

Entrega do Módulo de Pagamentos e Listagem de Endereços

Autor: Danilo Alexsander Oliveira Silva
Valnei Sousa Conceição Filho
Eric de Araújo Antunes

História do que foi solicitado

- Desenvolver duas funcionalidades essenciais para o sistema
- o Registro de Pagamentos e a Listagem de Endereços
- aplicando boas práticas de arquitetura, organização em camadas e divisão de responsabilidades entre os membros da equipe, garantindo um código limpo, funcional e preparado para evoluções futuras.

1. Registro e gerenciamento de pagamentos

2. Listagem de endereços do usuário

3. Estrutura completa seguindo camadas: ViewModel, DTOs, Controller e Service

5. Código organizado e versionado no GitHub

REGISTRO DE PAGAMENTO



- A funcionalidade solicitada foi o *registro de pagamento* no aplicativo.
- O objetivo era permitir que o usuário realizasse o pagamento e tivesse o registro salvo no sistema.
- Foi desenvolvido:
 - Uma **tela de registro**
 - Um **ViewModel** responsável por orquestrar os dados e comunicação com backend

Ator, Funcionalidade e Resultado

- **Ator:** Usuário do aplicativo
- **Funcionalidade:** Registrar pagamento informando valores e dados obrigatórios
- **Resultado:** Pagamento registrado com sucesso ou mensagem de erro

kotlin

```
data class RegistrarPagamentoRequest(  
    val valor: Double,  
    val metodo: String,  
    val descricao: String  
)  
  
data class RegistrarPagamentoResponse(  
    val status: String,  
    val mensagem: String  
)
```

Tela de Registro de Pagamento



- Telas criadas para registrar pagamento
- Campos básicos: valor, método, data
- Botão de envio
- Validação de campos

Registrar Pagamento

Valor

Método de Pagamento

Data

ViewModel do Registro de Pagamento



- ViewModel criado para processar o envio
- Comunicação com backend
- Tratamento de estado: carregando, sucesso, erro
- Lógica limpa e reativa

kotlin

```
class PagamentoViewModel : ViewModel() {  
  
    val estado = MutableLiveData<EstadoPagamento>()  
  
    fun registrarPagamento(request: RegistrarPagamentoRequest) {  
        estado.value = EstadoPagamento.Carregando  
  
        try {  
            val resposta = api.registrarPagamento(request)  
            estado.value = EstadoPagamento.Sucesso(resposta)  
        } catch (e: Exception) {  
            estado.value = EstadoPagamento.Erro("Falha ao registrar pagamento")  
        }  
    }  
}  
  
sealed class EstadoPagamento {  
    object Carregando : EstadoPagamento()  
    data class Sucesso(val data: RegistrarPagamentoResponse) : EstadoPagamento()  
    data class Erro(val mensagem: String) : EstadoPagamento()  
}
```

Casos de Exceção



- Falha na conexão
- Campos inválidos
- Backend retornando erro
- Mensagem exibida ao usuário

```
kotlin

if (valor <= 0) {
    throw IllegalArgumentException("Valor inválido")
}

catch (e: UnknownHostException) {
    return EstadoPagamento.Erro("Sem conexão com a internet")
}

catch (e: HttpException) {
    return EstadoPagamento.Erro("Erro no servidor: ${e.code()}")
}
```

LISTAGEM DE ENDEREÇOS



- O sistema precisava retornar para o usuário a **lista de endereços cadastrados**.
- A equipe dividiu a tarefa entre:
 - ErrosDTOs + EnderecoController
 - EnderecoMock + EnderecoService
 - DetalheErro, EnderecoDto, EnderecosResponse, ErroResponse
- Toda a implementação foi feita seguindo boas práticas de API REST.

Ator, Funcionalidade e Resultado

- **Ator:** Usuário logado autenticado com token
- **Funcionalidade:** Buscar todos os endereços através do endpoint `/api/v1/users/address`
- **Resultado esperado:**
 - Lista de endereços
 - Caso de erro → retorno estruturado (erro + detalhe)

kotlin

```
@GetMapping("/api/v1/users/address")
fun listarEnderecos(
    @RequestHeader("Authorization") token: String
): ResponseEntity<Any> {
    if (!token.startsWith("Bearer ")) {
        return ResponseEntity.status(401)
            .body(ErroResponse("Token inválido"))
    }

    return ResponseEntity.ok(service.listarEnderecos())
}
```

ErrosDTOs + EnderecoController



1. ErrosDTOs

- DTOs padrões para erros da API
- Facilita padronização e tratamento no front
- Ex.: ErroDTO, ErroValidacaoDTO

2. EnderecoController

- Recebe a requisição
- Valida o token
- Encaminha para o service do Erik
- Prepara a resposta JSON usando os DTOs

```
data class ErroDTO(val erro: String)
data class ErroValidacaoDTO(val campo: String, val mensagem: String)

@RestController
@RequestMapping("/api/v1/users/address")
class EnderecoController {

    @GetMapping
    fun listar(@RequestHeader("Authorization") token: String?): ResponseEntity<Any> {

        if (token.isNullOrEmpty() || !token.startsWith("Bearer ")) {
            return ResponseEntity.status(401).body(
                ErroDTO("Acesso não autorizado")
            )
        }

        return ResponseEntity.ok(service.listarEnderecos())
    }
}
```

EnderecoMock + EnderecoService

1. EnderecoMock

- Simula uma base de dados
- Utilizado para testar o comportamento da API

2. EnderecoService

- Recebe a chamada do controller
- Recupera dados do mock
- Aplica regras de negócio
- Retorna para o controller

kotlin

```
object EnderecoMock {  
    val lista = listOf(  
        EnderecoDto("Rua A", "123"),  
        EnderecoDto("Av B", "456")  
    )  
}  
  
@Service  
class EnderecoService {  
    fun listarEnderecos(): EnderecosResponse {  
        return EnderecosResponse(  
            total_enderecos = EnderecoMock.lista.size,  
            enderecos = EnderecoMock.lista  
        )  
    }  
}
```

DTOs e Respostas



DetalheErro → explica o motivo do erro

EnderecoDto → representa um único endereço

EnderecosResponse → retorno principal contendo:

- total_enderecos
- lista de endereços

ErroResponse → objeto de erro padronizado, usado pelo controller

```
kotlin

data class DetalheErro(
    val campo: String,
    val mensagem: String
)

data class EnderecoDto(
    val logradouro: String,
    val numero: String
)

data class EnderecosResponse(
    @JsonProperty("total_enderecos")
    val totalEnderecos: Int,
    val enderecos: List<EnderecoDto>
)

data class ErroResponse(
    val erro: String,
    val detalhes: List<DetalheErro>? = null
)
```

Tela / Resultado + Casos de Exceção



- Exemplo de sucesso

json

```
{
  "total_enderecos": 2,
  "enderecos": [
    { "rua": "Rua X", "numero": 40 },
    { "rua": "Av Y", "numero": 200 }
  ]
}
```

- Exemplo de sucesso

json

```
{
  "mensagem": "Token inválido",
  "detalhe": "O campo Authorization não foi informado"
}
```

Obrigados a todos!